جامعة خليفة
Khalifa University

NumPy (Numerical Python)

Dr. Sergio Martinez

Course layout

1. Introduction
2. Creating ndarrays
3. Indexing and slicing
4. Array manipulation
5. Math functions
6. Input/Outputs

1. Introduction

- **NumPy**
- Import the package
- ndarray object
- Data types
- Attributes

- **NumPy** (Numerical Python), is a library consisting of **multidimensional array objects** and a collection of routines for processing those arrays.
- Together with **Matplotlib** (plotting library), it can be used as a replacement for **Matlab**.
- Using Numpy, a developer can perform:
  - **Mathematical** and **logical** operations on arrays.
  - **Fourier transforms** and routines for shape manipulation.
  - Operations related to **linear algebra**.
- NumPy is **open source**.

1. Introduction

- NumPy
- **Import the package**
- ndarray object
- Data types
- Attributes

- **NumPy** is imported using the following syntax:

```
import numpy
```

- As a convention, we can import with:

```
import numpy as np
```

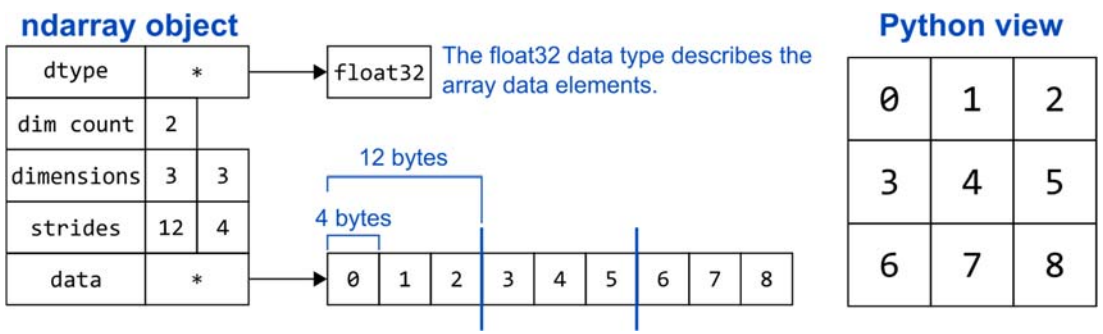
- Then, the usage will be:

```
np.command(xxx)
```

1. Introduction

- NumPy
- Import the package
- **ndarray object**
- Data types
- Attributes

- The most important object defined in **NumPy** is an **N-dimensional** array type called **ndarray**.
- It describes the collection of items of the **same type**.
- Items in the collection can be accessed using a **zero-based** index.



1. Introduction

- NumPy
- Import the package
- **ndarray object**
- Data types
- Attributes

- An instance of ndarray class can be constructed by different array creation routines described later

- The basic ndarray is created using an array function in NumPy as follows

```
import numpy as np
A = np.array
```

- It creates an ndarray from any object exposing array interface, or from any method that returns an array. The default constructor looks like:

```
A = np.array(object,
             dtype = None,
             copy = True,
             order = None,
             subok = False,
             ndmin = 0)
```

1. Introduction

- NumPy
- Import the package
- **ndarray object**
- Data types
- Attributes

- The previous constructor takes the following parameters:

Sr.No.	Parameter & Description
1	**object** Any object exposing the array interface method returns an array, or any (nested) sequence.
2	**dtype** Desired data type of array, optional
3	**copy** Optional. By default (true), the object is copied
4	**order** C (row major) or F (column major) or A (any) (default)
5	**subok** By default, returned array forced to be a base class array. If true, sub-classes passed through
6	**ndmin** Specifies minimum dimensions of resultant array

- **Let's see some examples:**

  01_Introduction/ndarray_object_constructor.ipynb

1. Introduction

- NumPy
- Import the package
- ndarray object
- **Data types**
- Attributes

- NumPy supports a much greater variety of **numerical types** than Python does. Some of them are:

Sr.No.	Data Types & Description
1	**bool_** Boolean (True or False) stored as a byte
2	**Int8, int16, int32, int64**
3	**uint8, uint16, uint32, uint64**
4	**float_, float16, float32, float64**
5	**complex_, complex64, complex128**

- Those numerical types are instances of **dtype** (data type) class and they can be used when we create ndarray objects using dtype parameter:

```
# dtype parameter
import numpy as np
a = np.array([1, 2, 3], dtype = np.complex_)
print(a)            # [1.+0.j 2.+0.j 3.+0.j]
print(a.dtype) # complex128
```

1. Introduction

- NumPy
- Import the package
- ndarray object
- **Data types**
- Attributes

- Alternatively, we can create custom dtype objects with structured data. We need to provide:
  - What are the names of the “fields” of the structure, by which they can be accessed.
  - What is the data-type of each field.
  - Which part of the memory block each field takes.
- **Let’s see some examples:**

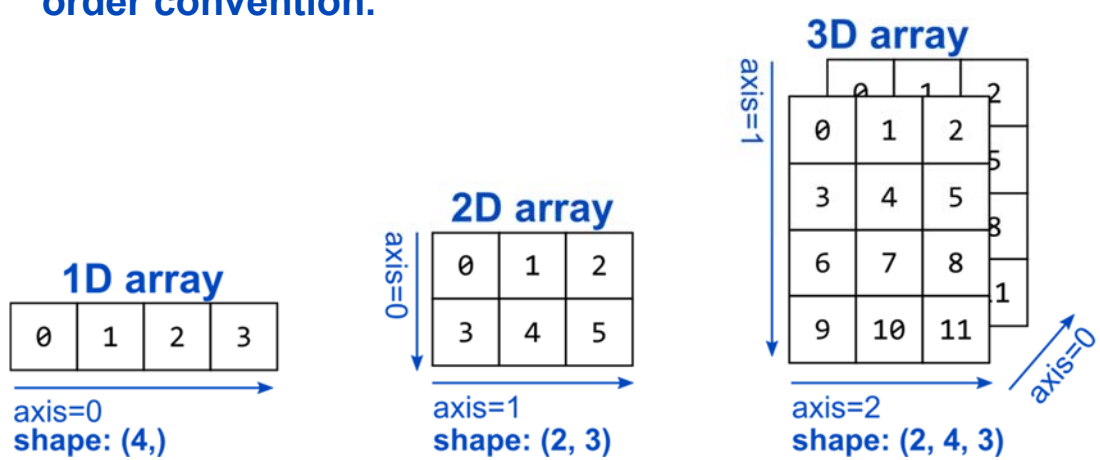
  01_Introduction/data_types.ipynb

1. Introduction

- NumPy
- Import the package
- ndarray object
- Data types
- **Attributes**

- The attribute **shape** of an array is a tuple indicating the number of elements among each axis. An existing array has this attribute **a.shape** which contains this tuple. It can be used to reshape an array.

- The **axes** of an array describe the order of indexing into the array, e.g., **axis=0** refers to the first index coordinate, **axis=1** refers to the second, etc. **Pay attention to the axis order convention.**



1. Introduction

- NumPy
- Import the package
- ndarray object
- Data types
- **Attributes**

- The attribute **ndim** of an array is a number indicating the number of dimensions of the array.

- The attribute **itemsize** of an array indicates the size of each element in the array in bytes

1D array

T	F	T	T

dtype: np.bool_
ndim: 1
itemsize: 1

2D array

0	1	2
3	4	5

dtype: np.int32
ndim: 2
itemsize: 4

3D array

-0.1	3.7	2.0
1.1	0.0	-1.7
6.2	0.0	8.7
0.0	9.9	-3.5

dtype: np.float64
ndim: 3
itemsize: 8

- **Let's see some examples:**

  01_Introduction/attributes.ipynb

2. Creating ndarrays

- **Array creation**
- np.empty()
- np.zeros()
- np.ones()
- np.asarray()
- np.arrange()

- A new **ndarray** object can be constructed by any of the following array creation routines or using a low-level ndarray constructor.
  - **np.array()**
  - **np.empty()**
  - **np.zeros()**
  - **np.ones()**
  - **np.asarray()**
  - **np.arange()**

2. Creating ndarrays

- Array creation
- **np.empty()**
- np.zeros()
- np.ones()
- np.asarray()
- np.arrange()

- **np.empty** creates an uninitialized array of specified shape and dtype. It uses the following constructor

```
np.empty(shape, dtype = float, order = 'C')
```

Sr.No.	Parameter & Description
1	**shape** Shape of an empty array in int or tuple of int
2	**dtype** By default, the data type of input data is applied to the resultant ndarray
3	**order** C (row major) or F (column major). C is default

2. Creating ndarrays

- Array creation
- np.empty()
- **np.zeros()**
- np.ones()
- np.asarray()
- np.arrange()

- **np.zeros** returns a new array of specified size, filled with zeros.

```
np.zeros(shape, dtype = float, order = 'C')
```

Sr.No.	Parameter & Description
1	**shape** Shape of an empty array in int or tuple of int
2	**dtype** By default, the data type of input data is applied to the resultant ndarray
3	**order** C (row major) or F (column major). C is default

2. Creating ndarrays

- Array creation
- np.empty()
- np.zeros()
- **np.ones()**
- np.asarray()
- np.arrange()

- **np.ones** returns a new array of specified size, filled with ones.

```
np.ones(shape, dtype = float, order = 'C')
```

Sr.No.	Parameter & Description
1	**shape** Shape of an empty array in int or tuple of int
2	**dtype** By default, the data type of input data is applied to the resultant ndarray
3	**order** C (row major) or F (column major). C is default

2. Creating ndarrays

- Array creation
- np.empty()
- np.zeros()
- np.ones()
- **np.asarray()**
- np.arrange()

- **np.asarray** This function is similar to np.array except for the fact that it has fewer parameters. This routine is useful for converting Python sequence into ndarray.

```
np.asarray(a, dtype = float, order = 'C')
```

Sr.No.	Parameter & Description
1	**a** Input data in any form such as list, list of tuples, tuples, tuple of tuples or tuple of lists
2	**dtype** By default, the data type of input data is applied to the resultant ndarray
3	**order** C (row major) or F (column major). C is default

2. Creating ndarrays

- Array creation
- np.empty()
- np.zeros()
- np.ones()
- np.asarray()
- **np.arrange()**

- **np.arange** returns an ndarray object containing evenly spaced values within a given range. The format of the function is as follows:

```
np.arange(start, stop, step, dtype)
```

Sr.No.	Parameter & Description
1	**start** The start of an interval. If omitted, defaults to 0
2	**stop** The end of an interval (not including this number)
3	**step** Spacing between values, default is 1
4	**dtype** Data type of resulting ndarray. If not given, data type of input is used

- **Let's see some examples:**

  02_Creating_ndarrays/ndarray_creation.ipynb

3. Indexing and slicing

- **Copies and views**
- 1D array
- 2D array
- 3D array
- Iterating over array
- Broadcasting

- New ndarray objects can also be created from another ndarray previously created. Creating a new ndarray by this way can be done as a **copy** or as a **view** of the existing one (or part of it when we use indexing and slicing).
- The main difference between a copy and a view of an array is that the **copy is a new array**, and the **view is just a view of the original array** (basically a pointer).
- The **copy owns the data** and any changes made to the copy will not affect original array, and any changes made to the original array will not affect the copy.
- The **view does not own the data** and any changes made to the view will affect the original array, and any changes made to the original array will affect the view.

3. Indexing and slicing

- **Copies and views**
- 1D array
- 2D array
- 3D array
- Iterating over array
- Broadcasting

- By default, assigning one ndarray to another will create a view.

```
import numpy as np

a = np.arange(10)
b = a
print(a)
print(b)

# a and b are the same object
print(id(a))
print(id(b))

# Making changes in b will also change a and viceversa
b[5] = 100
print(a)
print(b)
```

3. Indexing and slicing

- **Copies and views**
- 1D array
- 2D array
- 3D array
- Iterating over array
- Broadcasting

- A copy of the original ndarray can be done with the assignation using the function **np.copy()**.

```
import numpy as np

a = np.arange(10)
b = a.copy()
print(a)
print(b)

# a and b are different objects
print(id(a))
print(id(b))

# Making changes in b will not affect a
b[5] = 100
print(a)
print(b)
```

3. Indexing and slicing

- **Copies and views**
- 1D array
- 2D array
- 3D array
- Iterating over array
- Broadcasting

- A view of the original ndarray can also be done explicitely with the assignation using the function **np.view()**.

```
import numpy as np

a = np.arange(10)
b = a.view()
print(a)
print(b)
# a and b are the same object
print(id(a))
print(id(b))
# Making changes in b will also change a and viceversa
b[5] = 100
print(a)
print(b)
```

- **Let's see some examples:**

  03_Indexing_and_slicing/copies_and_views.ipynb

3. Indexing and slicing

- **Copies and views**
- 1D array
- 2D array
- 3D array
- Iterating over array
- Broadcasting

- The same approaches can be used when indexing or slicing ndarray objects.

- When the elements can be addressed with offsets, strides, and counts in the original array, the result will be given as a **view**:

```
import numpy as np
a = np.arange(10)
b = a[2:6] # b is a view of a
```

- When we use advanced indexing, the result will be given as a **copy**:

```
import numpy as np
a = np.arange(10)
b = a[[1,7]] # b is a copy of a elements
```

3. Indexing and slicing

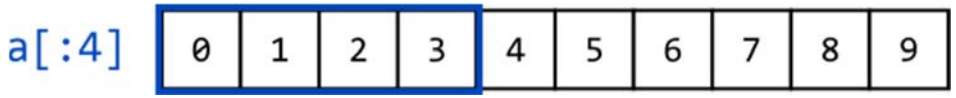
- Copies and views
- **1D array**
- 2D array
- 3D array
- Iterating over array
- Broadcasting

- **Indexing one element (view)**

3. Indexing and slicing

- Copies and views
- **1D array**
- 2D array
- 3D array
- Iterating over array
- Broadcasting

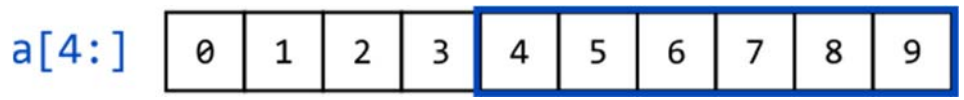
- **Slicing consecutive elements I (view)**



3. Indexing and slicing

- Copies and views
- **1D array**
- 2D array
- 3D array
- Iterating over array
- Broadcasting

- **Slicing consecutive elements II (view)**



3. Indexing and slicing

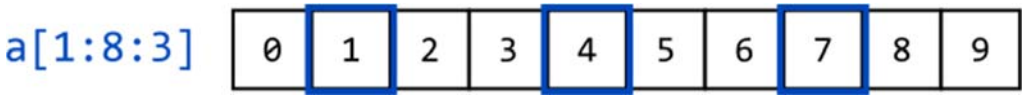
- Copies and views
- **1D array**
- 2D array
- 3D array
- Iterating over array
- Broadcasting

- **Slicing with range (view)**

3. Indexing and slicing

- Copies and views
- **1D array**
- 2D array
- 3D array
- Iterating over array
- Broadcasting

- **Slicing with range and stride I (view)**



3. Indexing and slicing

- Copies and views
- **1D array**
- 2D array
- 3D array
- Iterating over array
- Broadcasting

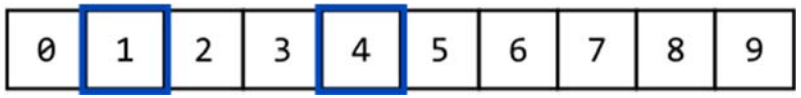
- **Slicing with range and stride II (view)**

3. Indexing and slicing

- Copies and views
- **1D array**
- 2D array
- 3D array
- Iterating over array
- Broadcasting

- **Advanced indexing (copy)**

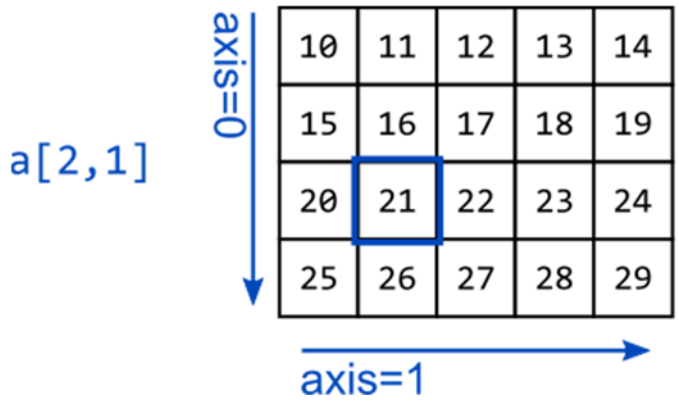
a[[1,4]]



3. Indexing and slicing

- Copies and views
- 1D array
- **2D array**
- 3D array
- Iterating over array
- Broadcasting

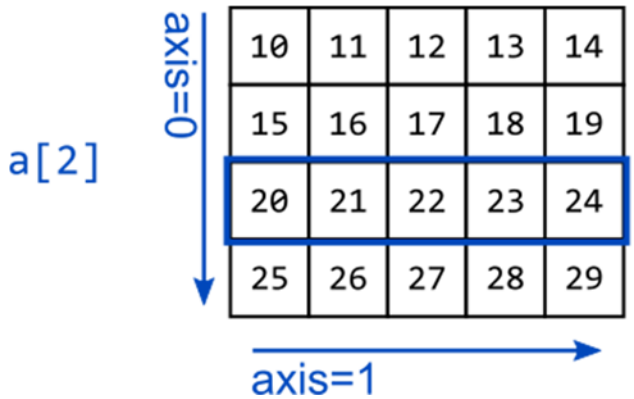
- **Indexing one element (view)**



3. Indexing and slicing

- Copies and views
- 1D array
- **2D array**
- 3D array
- Iterating over array
- Broadcasting

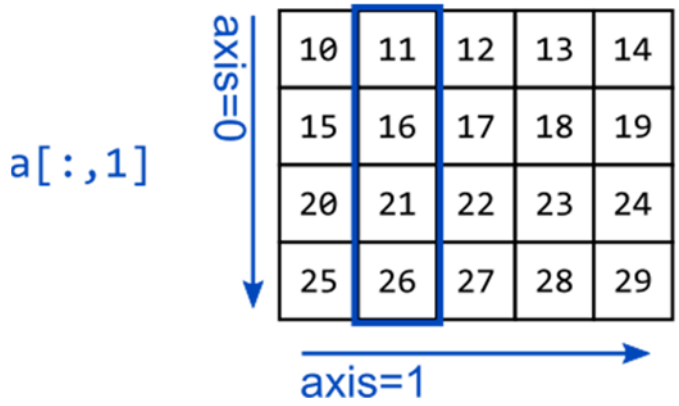
- **Slicing entire row (view)**



3. Indexing and slicing

- Copies and views
- 1D array
- **2D array**
- 3D array
- Iterating over array
- Broadcasting

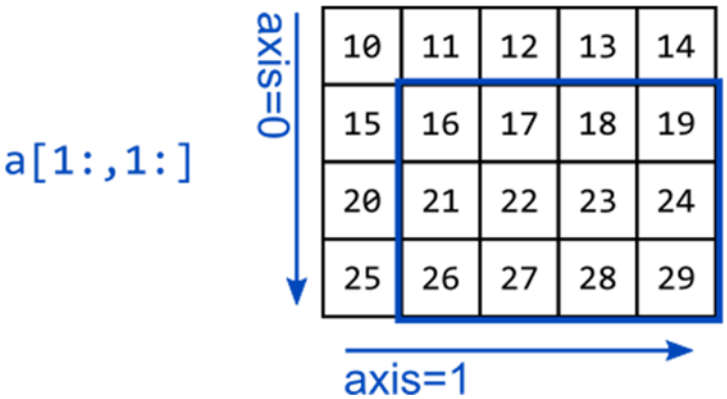
- **Slicing entire column (view)**



3. Indexing and slicing

- Copies and views
- 1D array
- **2D array**
- 3D array
- Iterating over array
- Broadcasting

- **Slicing sub-matrix (view)**

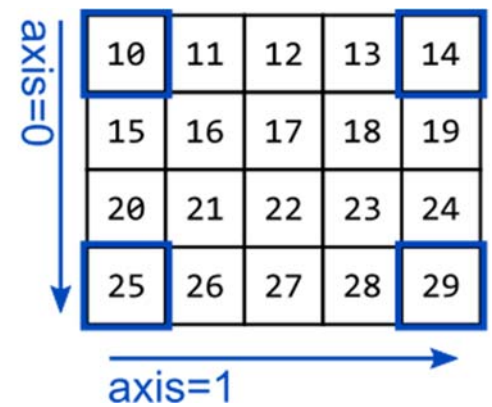


3. Indexing and slicing

- Copies and views
- 1D array
- **2D array**
- 3D array
- Iterating over array
- Broadcasting

- **Advanced indexing (copy)**

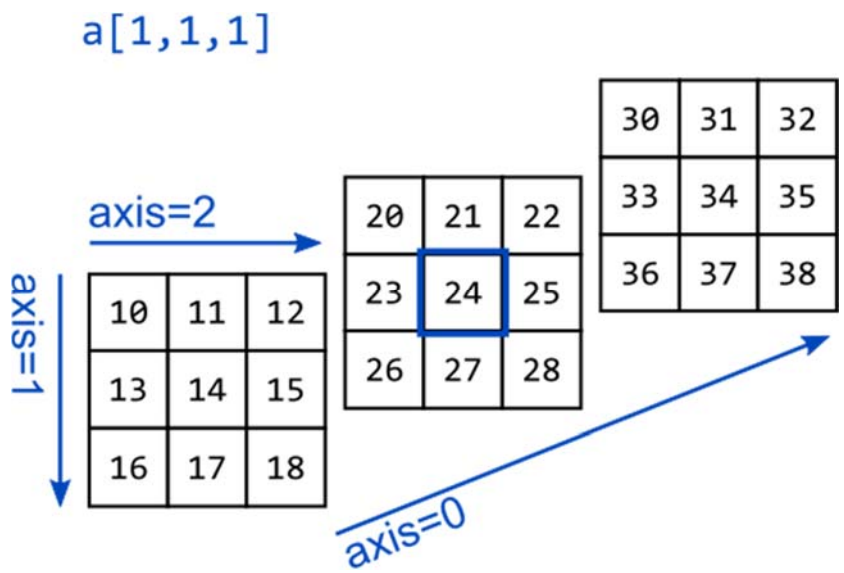
```
a[[0,0,3,3],
  [0,4,0,4]]
```



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- **3D array**
- Iterating over array
- Broadcasting

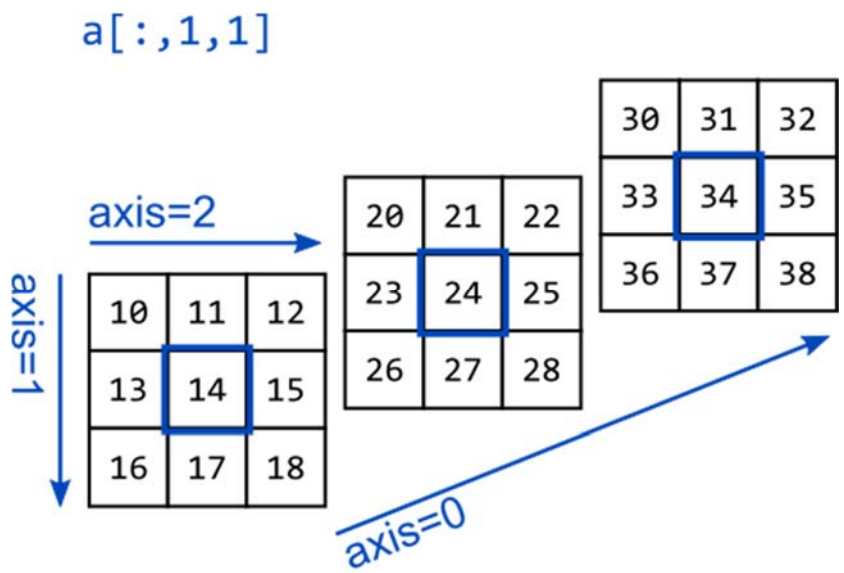
- **Indexing one element (view)**



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- **3D array**
- Iterating over array
- Broadcasting

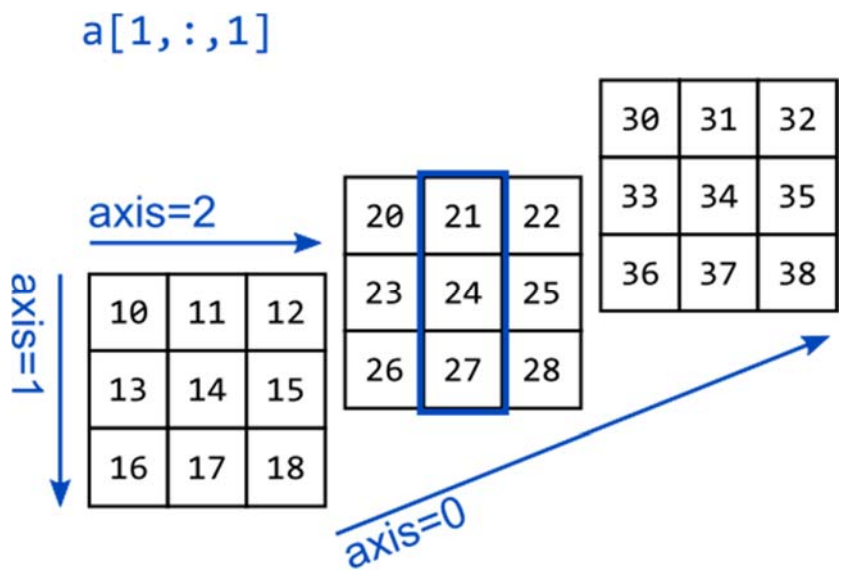
- **Slicing 1D array I (view)**



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- **3D array**
- Iterating over array
- Broadcasting

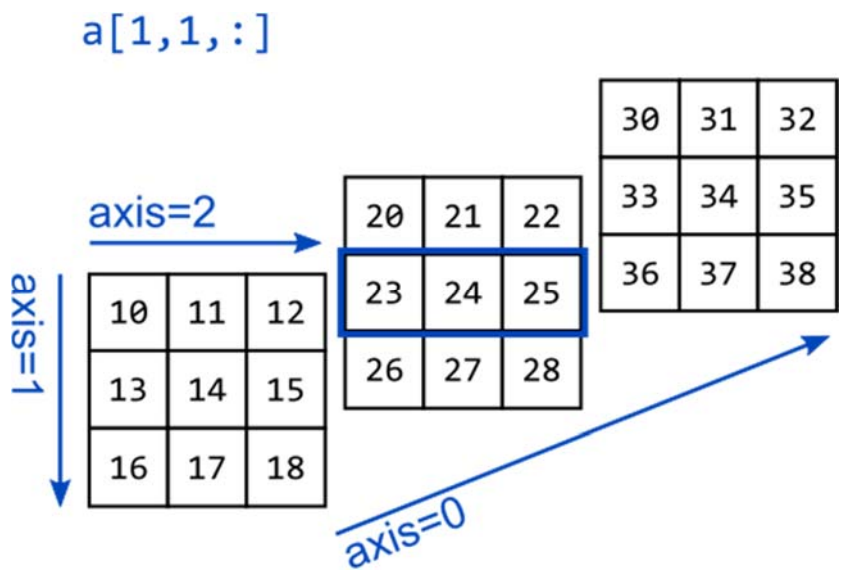
- **Slicing 1D array II (view)**



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- **3D array**
- Iterating over array
- Broadcasting

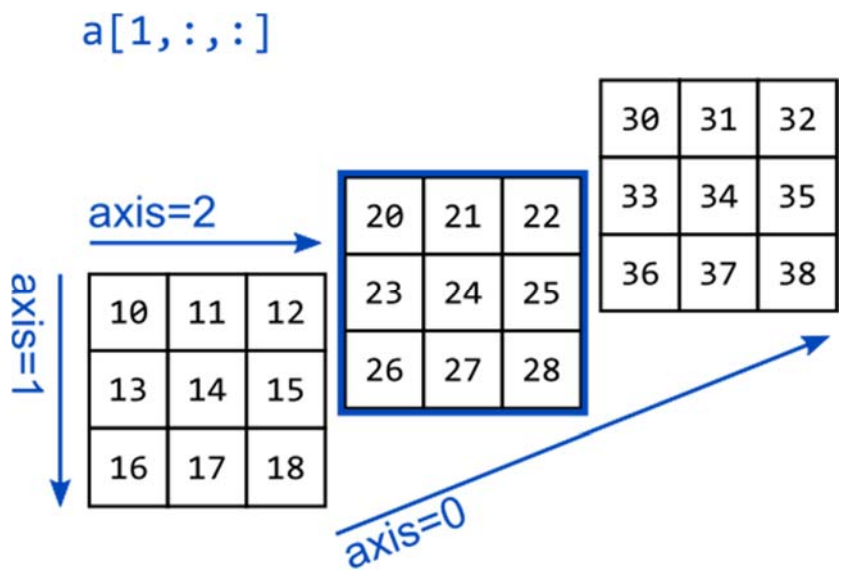
- **Slicing 1D array III (view)**



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- **3D array**
- Iterating over array
- Broadcasting

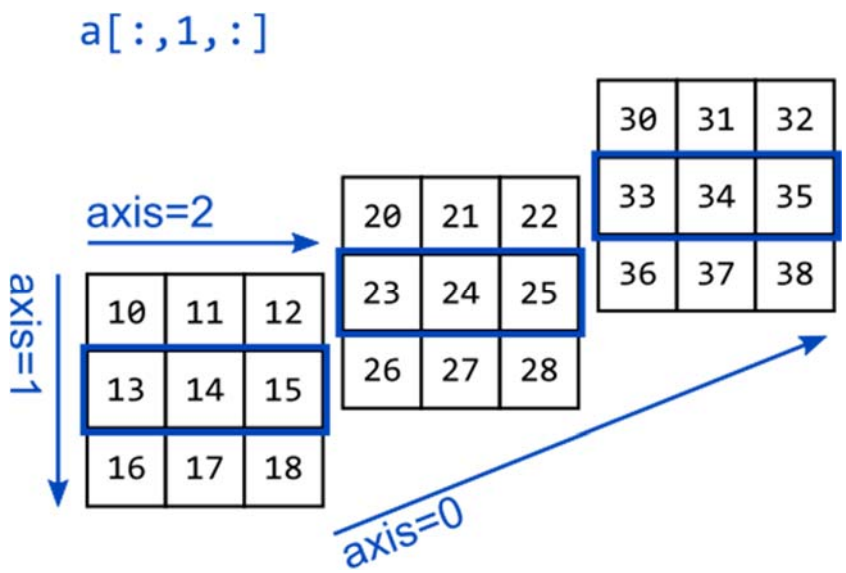
- **Slicing 2D array I (view)**



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- **3D array**
- Iterating over array
- Broadcasting

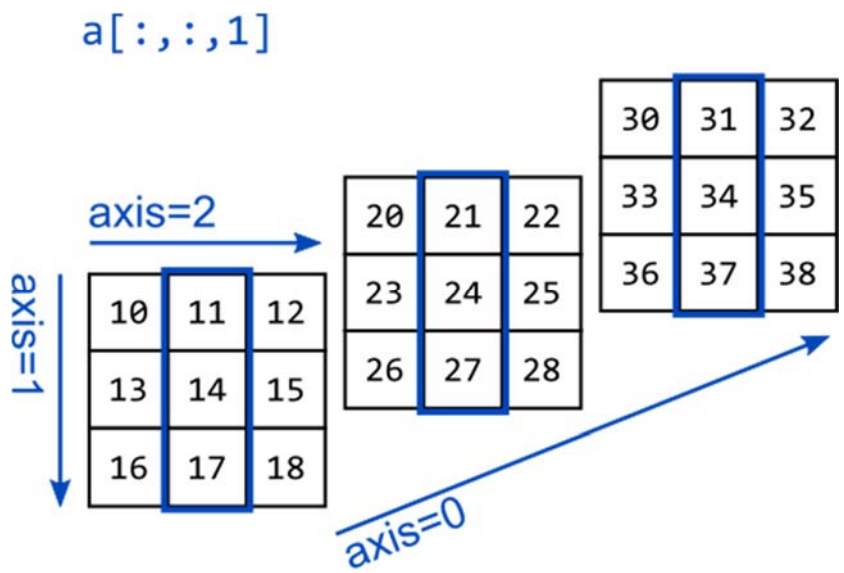
- **Slicing 2D array II (view)**



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- **3D array**
- Iterating over array
- Broadcasting

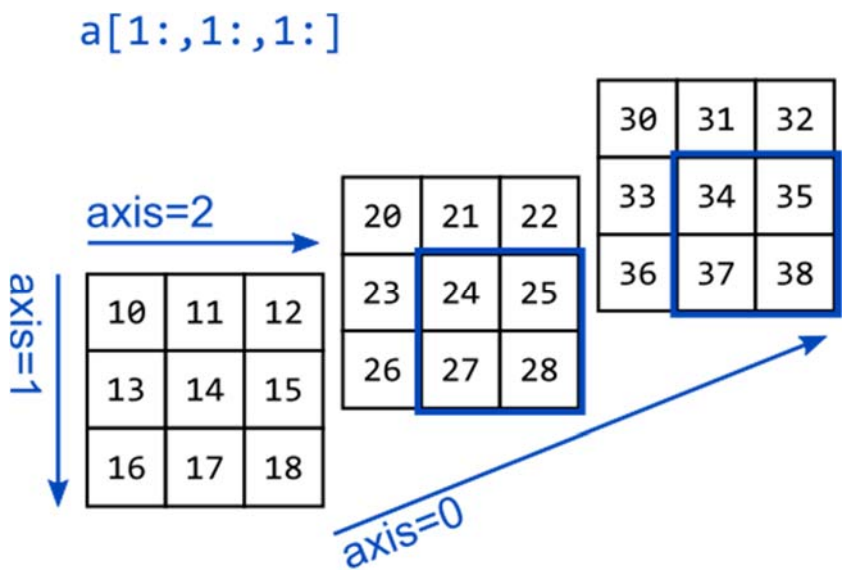
- **Slicing 2D array III (view)**



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- **3D array**
- Iterating over array
- Broadcasting

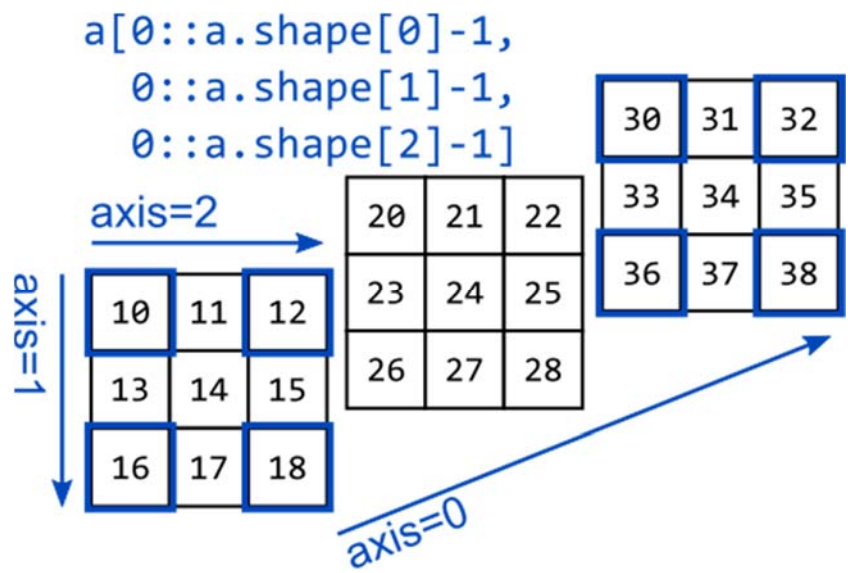
- **Slicing 3D array I (view)**



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- **3D array**
- Iterating over array
- Broadcasting

- **Slicing 3D array II (view)**



- **Let's see some examples:**

  03_Indexing_an_slicing/indexing_and_slicing.ipynb

3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- 3D array
- **Iterating over array**
- Broadcasting

- Iterating means going through elements one by one.
- As we deal with multi-dimensional arrays in numpy, we can do this using basic **for loop** of python.
- In basic for loops, iterating through each scalar of an ndarray we need to use **n for loops** which can be difficult to write for arrays with very high dimensionality. For example:

```
import numpy as np
a = np.array([[[1, 2, 3], [4, 5, 6]], [[7,
8, 9], [10, 11, 12]]])
for x in a:
    for y in x:
        for z in y:
            print(z)
```

3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- 3D array
- **Iterating over array**
- Broadcasting

- The iterator object **nditer** provides many flexible ways to visit all the elements of one or more arrays in a systematic fashion.

```
import numpy as np
a = np.array([[[1, 2, 3], [4, 5, 6]],
              [[7, 8, 9], [10, 11, 12]]])
for x in np.nditer(a):
    print(x)
```

- **Let's see some examples:**

  03_Indexing_and_slicing/iterating_over_array.ipynb

3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- 3D array
- Iterating over array
- **Broadcasting**

- numpy operations are usually done element-by-element which requires two arrays to have exactly the same shape.

```
import numpy as np
a = np.array([1.0, 2.0, 3.0])
b = np.array([2.0, 2.0, 2.0])
c = a * b
print(c) # [2. 4. 6.]
```

- The term **broadcasting** describes how numpy treats arrays with different shapes during **arithmetic operations**. Subject to certain constraints, the smaller array is “broadcast” across the larger array so that they have compatible shapes.

```
import numpy as np
a = np.array([1.0, 2.0, 3.0])
b = 2.0
c = a * b
print(c) # [2. 4. 6.]
```

3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- 3D array
- Iterating over array
- **Broadcasting**

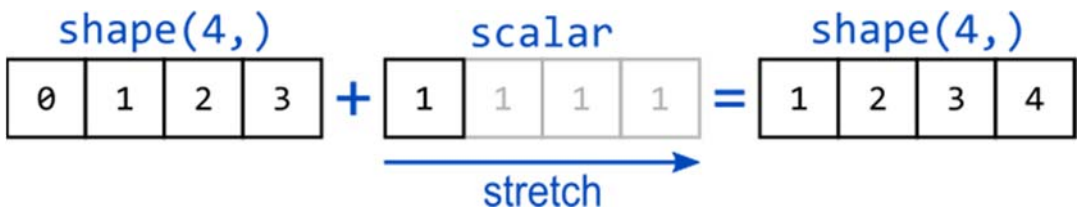
- **Broadcasting rules:**
  - If arrays have different number of dimensions, **insert new axis on the left** until arrays have the same number of dimensions.
  - For each axis i, if arrays dimension[i] do not match, **"stretch"** the arrays where dimension[i] = 1 **to match other arrays**.
  - If no match and dimension[i] != 1, then there will be an error.

```
ValueError: operands could not be broadcast together with shapes …
```

3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- 3D array
- Iterating over array
- **Broadcasting**

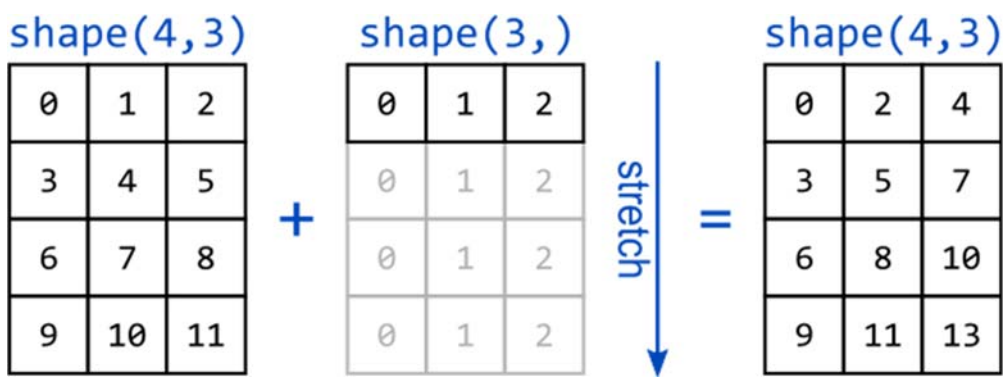
- **Broadcasting - 1D array and scalar:**



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- 3D array
- Iterating over array
- **Broadcasting**

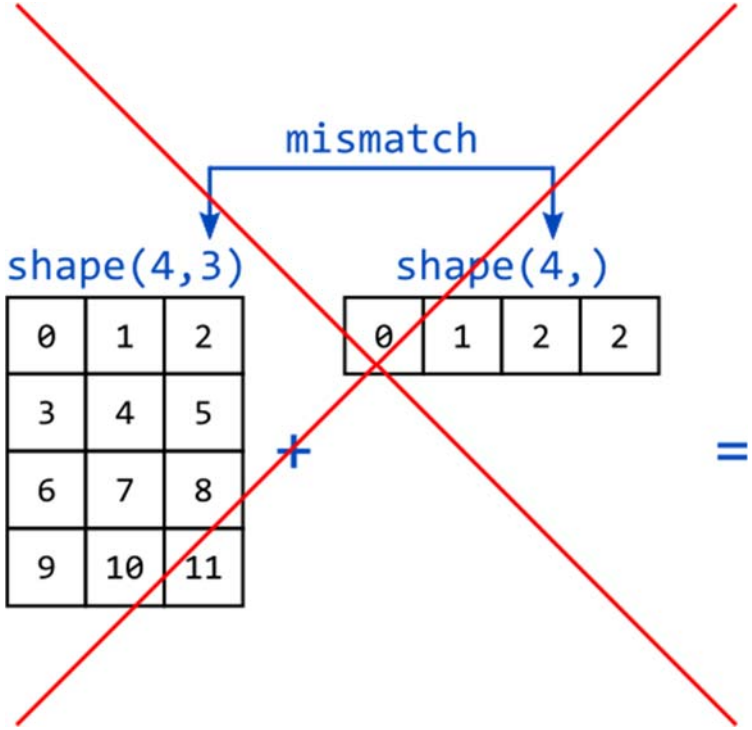
- **Broadcasting - 2D array and 1D array I:**



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- 3D array
- Iterating over array
- **Broadcasting**

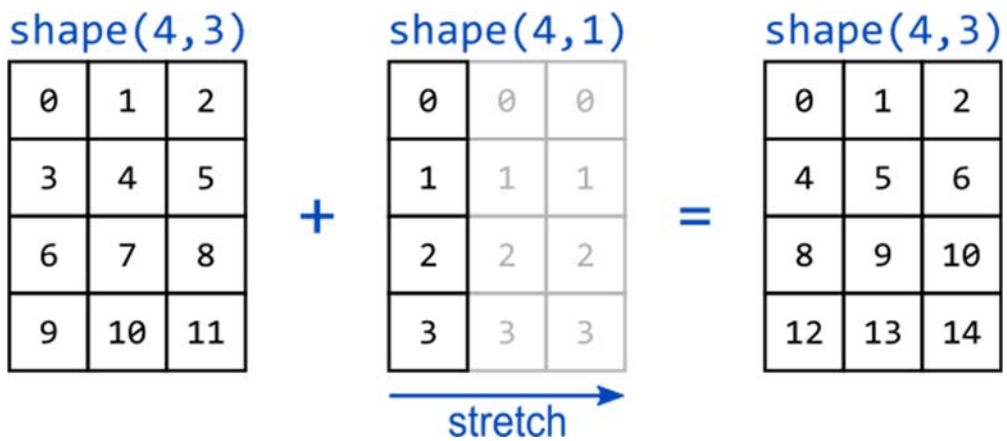
- **Broadcasting - 2D array and 1D array II:**



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- 3D array
- Iterating over array
- **Broadcasting**

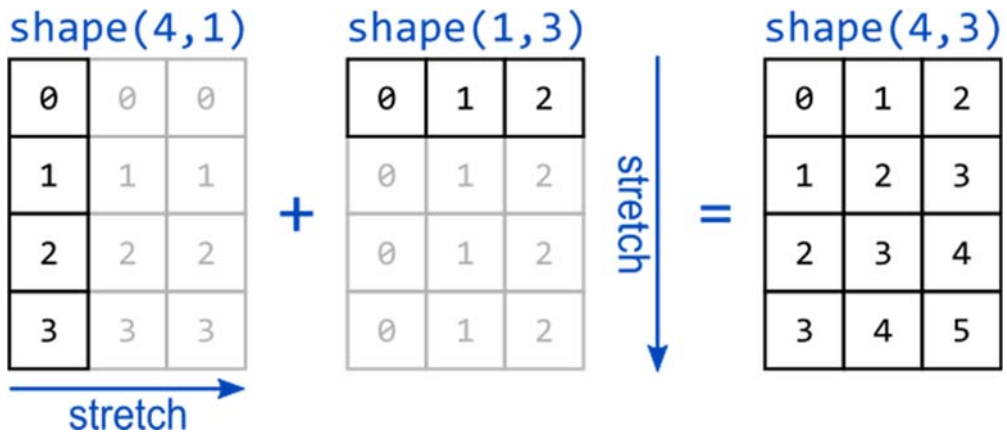
- **Broadcasting - 2D array and 2D array I:**



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- 3D array
- Iterating over array
- **Broadcasting**

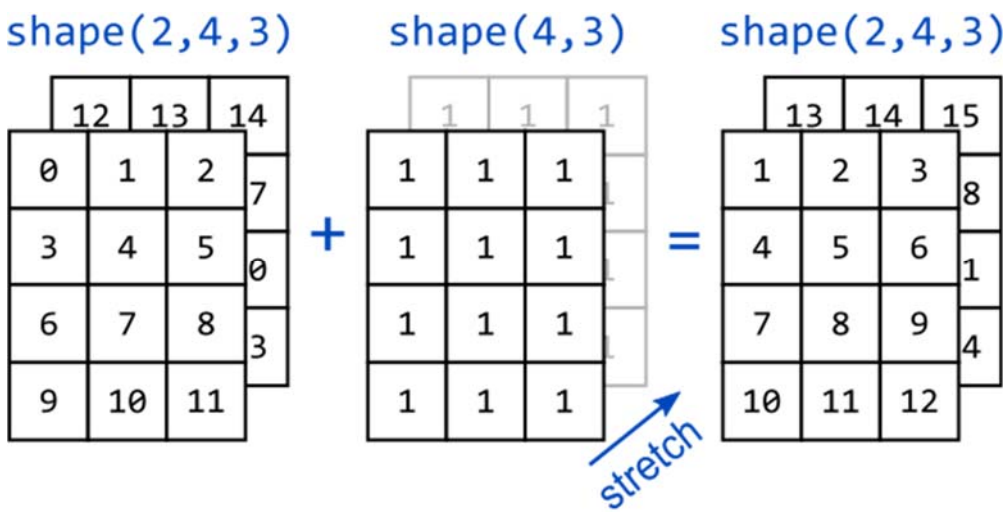
- **Broadcasting - 2D array and 2D array II:**



3. Indexing and slicing

- Copies and views
- 1D array
- 2D array
- 3D array
- Iterating over array
- **Broadcasting**

- **Broadcasting - 3D array and 2D array:**



- **Let's see some examples:**

  03_Indexing_and_slicing/broadcasting.ipynb

4. Array manipulation

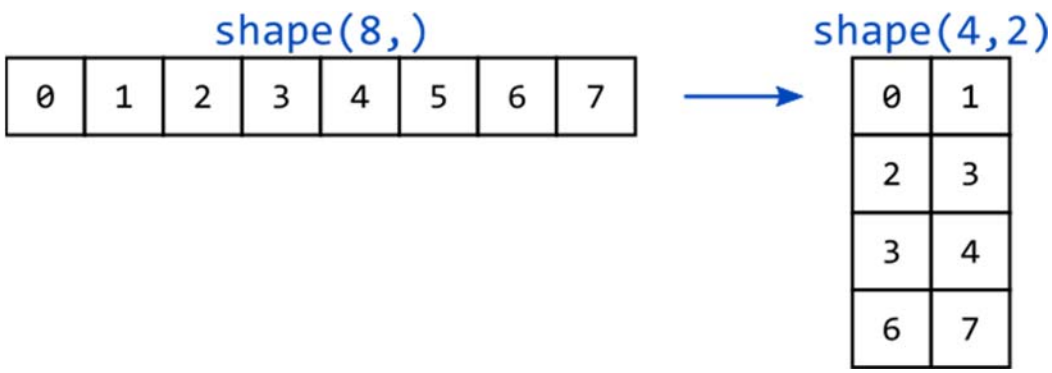
- **Changing shape**
- Transpose
- Change dimensions
- Joining arrays
- Splitting arrays
- Adding/Removing elements

Sr.No.	Function & Description
1	**reshape** Gives a new shape to an array without changing its data.
2	**flat** A 1-D iterator over the array
3	**flatten** Returns a copy of the array collapsed into one dimension
4	**ravel** Returns a contiguous flattened array

4. Array manipulation

- **Changing shape**
- Transpose
- Change dimensions
- Joining arrays
- Splitting arrays
- Adding/Removing elements

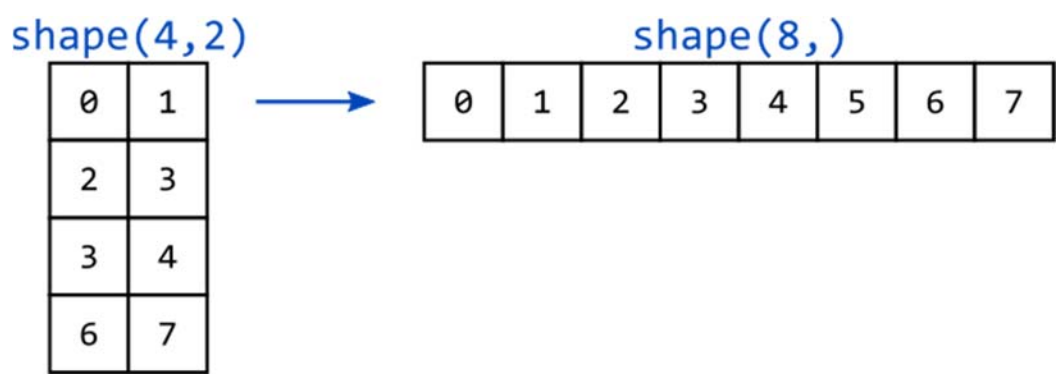
- **Reshape 1D array into 2D array (view):**



4. Array manipulation

- **Changing shape**
- Transpose
- Change dimensions
- Joining arrays
- Splitting arrays
- Adding/Removing elements

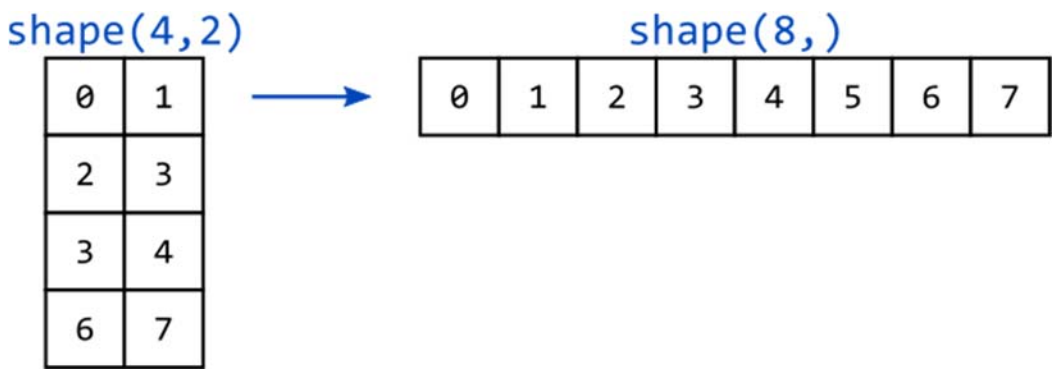
- **Flatten 2D array (copy):**



4. Array manipulation

- **Changing shape**
- Transpose
- Change dimensions
- Joining arrays
- Splitting arrays
- Adding/Removing elements

- **Ravel 2D array (view):**



- **Let's see some examples:**

  04_Array_manipulation/changing_shape.ipynb

4. Array manipulation

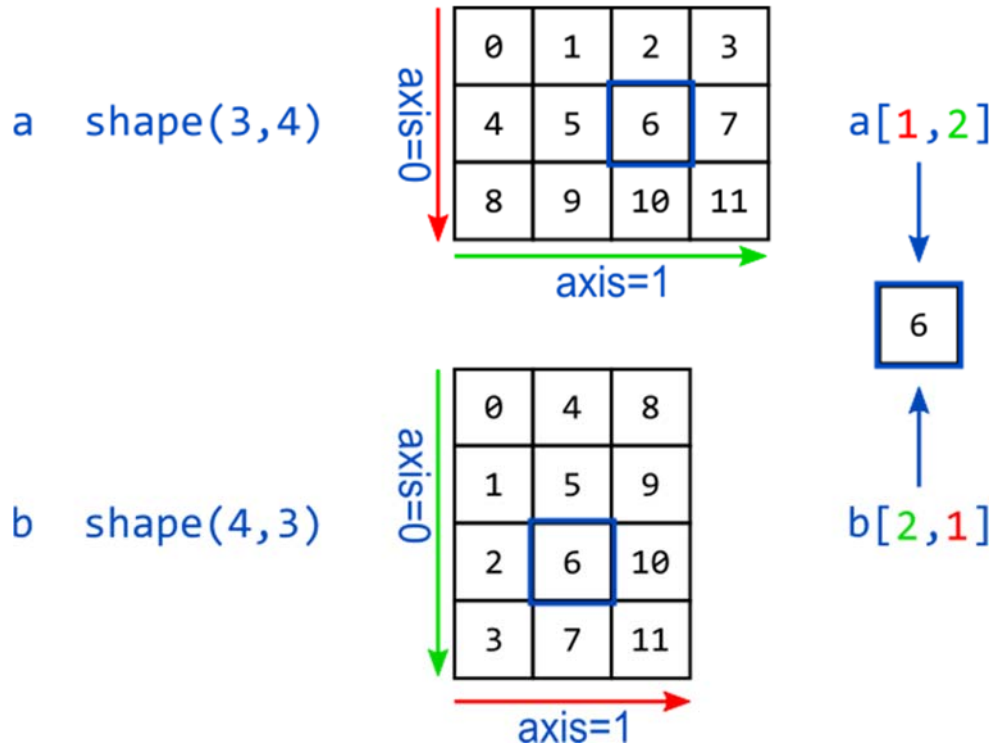
- Changing shape
- **Transpose**
- Change dimensions
- Joining arrays
- Splitting arrays
- Adding/Removing elements

Sr.No.	Function & Description
1	**transpose** Permutes the dimensions of an array
2	**T** Same as self.transpose()
3	**rollaxis** Rolls the specified axis backwards
4	**swapaxes** Interchanges the two axes of an array

4. Array manipulation

- Changing shape
- **Transpose**
- Change dimensions
- Joining arrays
- Splitting arrays
- Adding/Removing elements

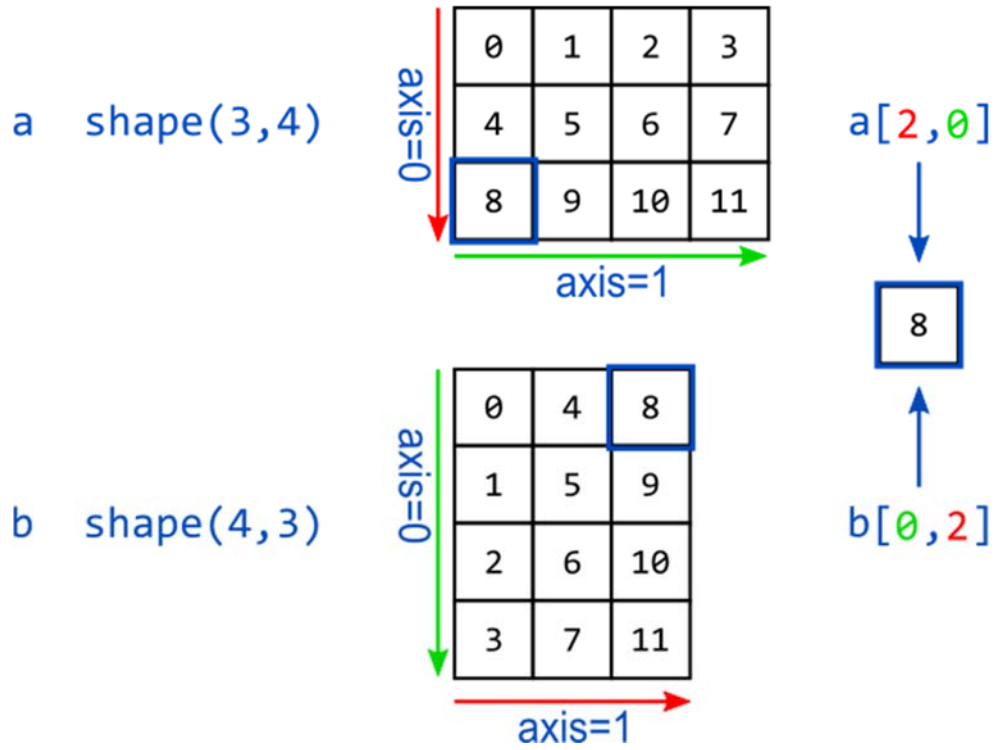
- **Transpose of 2D array I (view):**



4. Array manipulation

- Changing shape
- **Transpose**
- Change dimensions
- Joining arrays
- Splitting arrays
- Adding/Removing elements

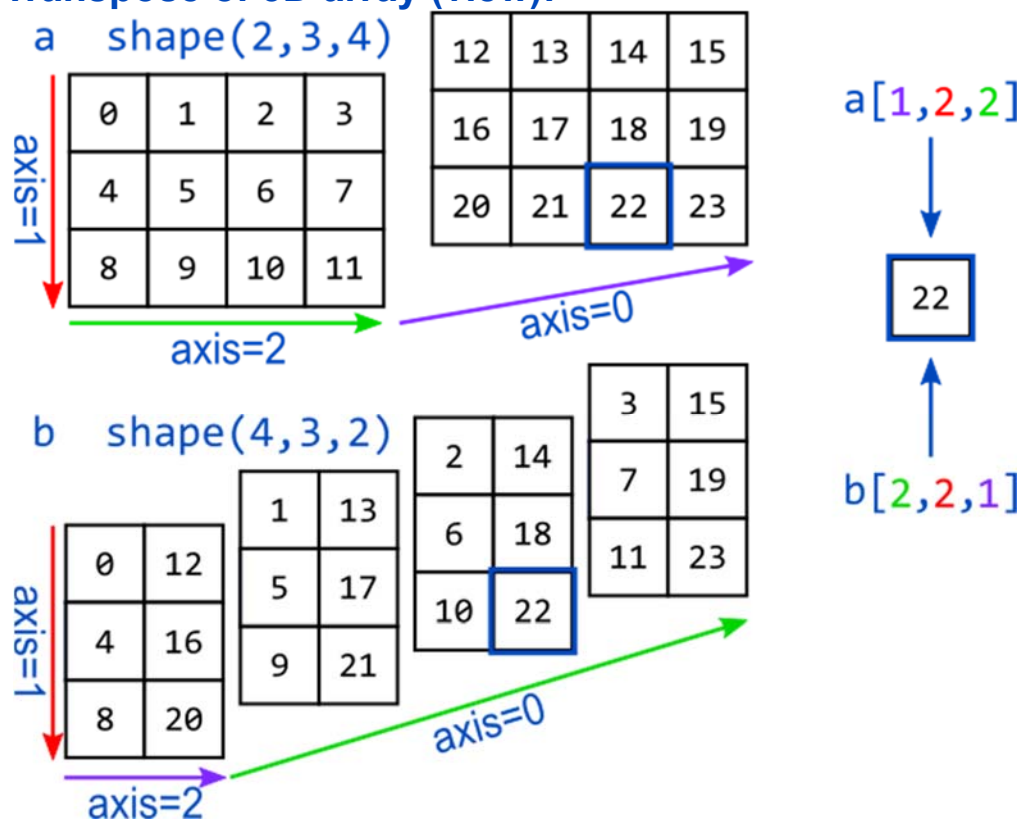
- **Transpose of 2D array II (view):**



4. Array manipulation

- Changing shape
- **Transpose**
- Change dimensions
- Joining arrays
- Splitting arrays
- Adding/Removing elements

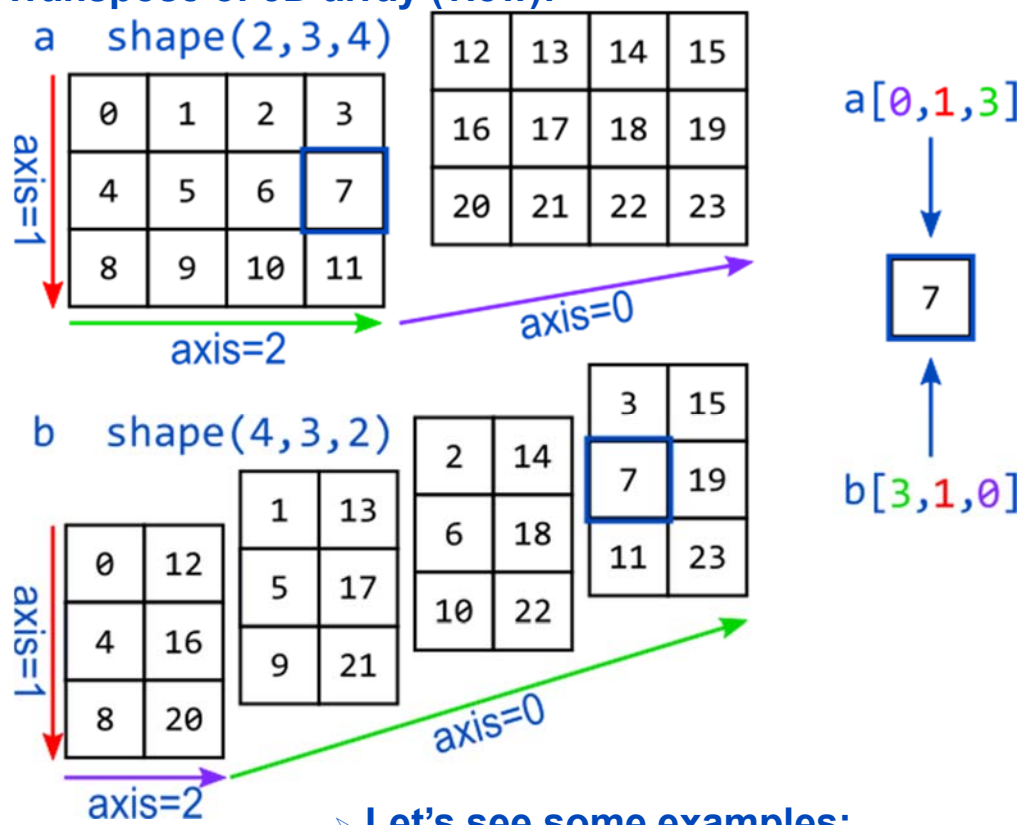
- **Transpose of 3D array (view):**



4. Array manipulation

- Changing shape
- **Transpose**
- Change dimensions
- Joining arrays
- Splitting arrays
- Adding/Removing elements

- **Transpose of 3D array (view):**



- **Let's see some examples:**

04_Array_manipulation/transpose.ipynb

4. Array manipulation

- Changing shape
- Transpose
- **Change dimensions**
- Joining arrays
- Splitting arrays
- Adding/Removing elements

Sr.No.	Function & Description
1	**broadcast** Produces an object that mimics broadcasting
2	**broadcast_to** Broadcasts an array to a new shape
3	**expand_dims** Expands the shape of an array
4	**squeeze** Removes single-dimensional entries from the shape of an array

- **Let's see some examples:**

  04_Array_manipulation/changing_dimensions.ipynb

4. Array manipulation

- Changing shape
- Transpose
- Change dimensions
- **Joining arrays**
- Splitting arrays
- Adding/Removing elements

Sr.No.	Function & Description
1	**concatenate** Joins a sequence of arrays along an existing axis
2	**stack** Joins a sequence of arrays along a new axis
3	**hstack** Stacks arrays in sequence horizontally (column wise)
4	**vstack** Stacks arrays in sequence vertically (row wise)

- **Let's see some examples:**

  04_Array_manipulation/joining_arrays.ipynb

4. Array manipulation

- Changing shape
- Transpose
- Change dimensions
- Joining arrays
- **Splitting arrays**
- Adding/Removing elements

Sr.No.	Function & Description
1	**split** Splits an array into multiple sub-arrays
2	**hsplit** Splits an array into multiple sub-arrays horizontally (column-wise)
3	**vsplit** Splits an array into multiple sub-arrays vertically (row-wise)

- **Let's see some examples:**

  04_Array_manipulation/splitting_arrays.ipynb

4. Array manipulation

- Changing shape
- Transpose
- Change dimensions
- Joining arrays
- Splitting arrays
- **Adding/Removing elements**

Sr.No.	Function & Description
1	**resize** Returns a new array with the specified shape
2	**append** Appends the values to the end of an array
3	**insert** Inserts the values along the given axis before the given indices
4	**delete** Returns a new array with sub-arrays along an axis deleted
5	**unique** Finds the unique elements of an array

- **Let's see some examples:**

  04_Array_manipulation/adding_removing.ipynb

5. Math functions

- **Rounding functions**
- Arithmetical functions
- Statistical functions
- Linear algebra

Sr.No.	Function & Description
1	**around** Returns the value rounded to the desired precision.
2	**floor** This function returns the largest integer not greater than the input parameter.
3	**ceil** This function returns the smallest integer greater than the input parameter.

- **Let's see some examples:**

  05_Math_functions/rounding_functions.ipynb

5. Math functions

- Rounding functions
- **Arithmetical functions**
- Statistical functions
- Linear algebra

Sr.No.	Function & Description
1	**add** Adds arguments element-wise.
2	**substract** Substracts arguments element-wise.
3	**multiply** Multiplies arguments element-wise.
4	**divide** Divides arguments element-wise.
5	**power** First array elements raised to powers from second array, element-wise.
6	**mod** Return element-wise remainder of division.

- **Let's see some examples:**

  05_Math_functions/arithmetical_functions.ipynb

5. Math functions

- Rounding functions
- Arithmetical functions
- **Statistical functions**
- Linear algebra

Sr.No.	Function & Description
1	**amax** Returns the maximum of an array or maximum along an axis.
2	**amin** Returns the minimum of an array or minimum along an axis.
3	**sum** Sum of array elements over a given axis.
4	**mean** Returns the arithmetic mean of elements in the array.
5	**average** Computes the weighted average of elements in an array according to their respective weight given in another array.

- **Let's see some examples:**

  05_Math_functions/statistical_functions.ipynb

5. Math functions

- Rounding functions
- Arithmetical functions
- Statistical functions
- **Linear algebra**

Sr.No.	Function & Description
1	**dot** Dot product of two arrays.
2	**matmul** Matrix multiplication.
3	**determinant** Computes the determinant of the array.
4	**inv** Finds the inverse of a matrix.

- **Let's see some examples:**

  05_Math_functions/linear_algebra.ipynb

6. Input/output

- **Save / Load**
- Savez / Load
- Load image

- ndarray objects can be **saved** to and **loaded** from the disk files.

- NumPy introduces a simple file format for ndarray objects. This **.npy** file stores data, shape, dtype and other information required to reconstruct the ndarray in a disk file.

```
import numpy as np
a = np.array([1,2,3,4,5])
np.save('outfile',a)
b = np.load('outfile.npy')
print(b) # array([1, 2, 3, 4, 5])
```

6. Input/output

- Save / Load
- **Savez / Load**
- Load image

- It is possible to save multiple ndarray objects in the same file **.npz** using the finction **savez()**.

```
import numpy as np
a = np.array([1,2,3,4,5])
b = np.arange(12).reshape(3,4)
c = np.arange(24).reshape(2,3,4)
np.savez('workspace', a_name=a, b_name=b,
c_name=c)

npz_file = np.load('workspace.npz')
print(npz_file.files, '\n')
print('a = ', npz_file['a_name'], '\n')
print('b = ', npz_file['b_name'], '\n')
print('c = ', npz_file['c_name'], '\n')
```

- **Let's see some examples:**

  06_Input_output/save_load.ipynb

6. Input/output

- Save / Load
- Savez / Load
- **Load image**

- This is one example of how to **import an image from disk** into a numpy object using **skimage** package.

- Generally, an image will be a **3D array** where one of the dimensions will have the value of 3, corresponding to the values of the light in the red, green and blue wavelengths **(RGB)**.

```
from skimage import io
im= io.imread("lena_color.jpg")

print(type(im)) # <class 'numpy.ndarray'>
print(im.dtype) # uint8
print(im.shape) # (512, 512, 3)
```

- **Let's see some examples:**

  06_Input_output/load_image.ipynb

جامعة خليفة
Khalifa University

Thank You

ku.ac.ae